0.34. Un carré autour d'une lettre, avec carre

Une ligne, une lettre : carre(10, 8, ALPHABET[0], 1, 1, 250, 1) fait faire au A un tour en carré autour de la case (10, 8).

1. Ce qu'il faut comprendre

Une seule lettre, une seule ligne. Le A fait **un tour complet** autour de la case (10, 8), en carré, puis revient à sa place.

Ce qui est nouveau ici : `carre(x, y, tuile, taille, sens, vitesse, tours)`. Elle a sept réglages, mais pour commencer, ne regarde que les **trois premiers** :

- `10, 8` (`x`, `y`) : la case **autour de laquelle** on tourne, le **centre** du carré. La lettre part de là, et y revient.
- `ALPHABET[0]` (`tuile`) : ce qui tourne, la lettre A.

Les quatre autres restent simples ici : taille `1`, sens `1`, vitesse `250`, tours `1`. Ils veulent dire : le plus petit carré, dans le sens des aiguilles d'une montre, 4 pas par seconde, un seul tour. Les cours suivants les changent **un par un**.

Le trajet : le A part du centre (10, 8), fait un pas en diagonale jusqu'au coin en haut à gauche (9, 7), fait le tour du carré de **3 × 3** cases (droite, bas, gauche, haut), puis revient en diagonale au centre.

`carre` **ne rend rien :** la lettre finit exactement là où elle a commencé. Il n'y a ni `x =` ni variable à tenir.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <carre>` → 0.33.3. La fonction carre() — tourner en carré
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// main : le programme commence ici. Les lignes entre { et } s'exécutent`	Un commentaire : une phrase pour celui qui lit (« main : le programme commence ici. Les lignes entre { et } s'exécutent »). La console l'ignore.
`// dans l'ordre, de haut en bas.`	Un commentaire : une phrase pour celui qui lit (« dans l'ordre, de haut en bas. »). La console l'ignore.
`#include <carre>  // une tuile qui tourne en carré`	Demande à la console « carre » (ce que c'est : une tuile qui tourne en carré). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « une tuile qui tourne en carré ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`// Une seule ligne fait tout le carré. Ses sept réglages, dans l'ordre :`	Un commentaire : une phrase pour celui qui lit (« Une seule ligne fait tout le carré. Ses sept réglages, dans l'ordre : »). La console

	l'ignore.
`//  carre(10, 8, ALPHABET[0], 1, 1, 250, 1);`	Un commentaire : une phrase pour celui qui lit (« carre(10, 8, ALPHABET[0], 1, 1, 250, 1); »). La console l'ignore.
`//  |  |  |  |  |  |  |`	Un commentaire : une phrase pour celui qui lit (« ||||||| »). La console l'ignore.
`//  |  |  |  |  |  |  +-- tours : 1, un seul tour`	Un commentaire : une phrase pour celui qui lit (« ||||| +-- tours : 1, un seul tour »). La console l'ignore.
`//  |  |  |  |  |  +------- vitesse : 250 millisecondes par pas,`	Un commentaire : une phrase pour celui qui lit (« |||| +------- vitesse : 250 millisecondes par pas, »). La console l'ignore.
`//  |  |  |  |  |  soit 4 pas par seconde`	Un commentaire : une phrase pour celui qui lit (« |||| soit 4 pas par seconde »). La console l'ignore.
`//  |  |  |  |  +---------- sens : 1, comme les aiguilles`	Un commentaire : une phrase pour celui qui lit (« ||| +---------- sens : 1, comme les aiguilles »). La console l'ignore.
`//  |  |  |  |  d'une montre`	Un commentaire : une phrase pour celui qui lit (« ||| d'une montre »). La console l'ignore.
`//  |  |  |  +------------ taille : 1, le plus petit carré (3 × 3)`	Un commentaire : une phrase pour celui qui lit (« || +------------ taille : 1, le plus petit carré (3 × 3) »). La console l'ignore.
`//  |  |  +------------------------- tuile : ce qui tourne ; ALPHABET[0]`	Un commentaire : une phrase pour celui qui lit (« | +------------------------- tuile : ce qui tourne ; ALPHABET[0] »). La console l'ignore.
`//  |  |  est la 1re lettre, A`	Un commentaire : une phrase pour celui qui lit (« || est la 1re lettre, A »). La console l'ignore.
`//  |  +---------------------------- y : la LIGNE du centre, 8`	Un commentaire : une phrase pour celui qui lit (« | +---------------------------- y : la LIGNE du centre, 8 »). La console l'ignore.
`//  +------------------------------- x : la COLONNE du centre, 10`	Un commentaire : une phrase pour celui qui lit (« +------------------------------- x : la COLONNE du centre, 10 »). La console l'ignore.
`// Pour ce premier cours, regarde surtout x, y et la tuile. Les quatre`	Un commentaire : une phrase pour celui qui lit (« Pour ce premier cours, regarde surtout x, y et la tuile. Les quatre »). La console l'ignore.
`// autres réglages seront changés un par un dans les cours suivants.`	Un commentaire : une phrase pour celui qui lit (« autres réglages seront changés un par un dans les cours suivants. »). La console l'ignore.
`// Ce que fait le A, un pas tous les quarts de seconde :`	Un commentaire : une phrase pour celui qui lit (« Ce que fait le A, un pas tous les quarts de seconde : »). La console l'ignore.
`//  colonne :  9  10  11`	Un commentaire : une phrase pour celui qui lit (« colonne : 9 10 11 »). La console l'ignore.
`//  ligne 7 :  2 → 3 → 4`	Un commentaire : une phrase pour celui qui lit (« ligne 7 : 2 → 3 → 4 »). La console l'ignore.
`//  ↑  ↓`	Un commentaire : une phrase pour celui qui lit (« ↑ ↓ »). La console l'ignore.
`//  ligne 8 :  9  A  5`	Un commentaire : une phrase pour celui qui lit (« ligne 8 : 9 A 5 »). La console l'ignore.
`//  ↑  ↓`	Un commentaire : une phrase pour celui qui lit (« ↑ ↓ »). La console l'ignore.

`//  ligne 9 :  8 ← 7 ← 6`	Un commentaire : une phrase pour celui qui lit (« ligne 9 : 8 ← 7 ← 6 »). La console l'ignore.
`//  1.  le A apparaît au centre (10, 8)`	Un commentaire : une phrase pour celui qui lit (« 1. le A apparaît au centre (10, 8) »). La console l'ignore.
`//  2.  un pas en diagonale : le coin en haut à gauche (9, 7)`	Un commentaire : une phrase pour celui qui lit (« 2. un pas en diagonale : le coin en haut à gauche (9, 7) »). La console l'ignore.
`//  3 à 9. le tour, une case à la fois : droite, bas, gauche, haut…`	Un commentaire : une phrase pour celui qui lit (« 3 à 9. le tour, une case à la fois : droite, bas, gauche, haut… »). La console l'ignore.
`//  10.  …jusqu'au coin (9, 7), d'où il était parti`	Un commentaire : une phrase pour celui qui lit (« 10. …jusqu'au coin (9, 7), d'où il était parti »). La console l'ignore.
`//  11.  un pas en diagonale : retour au centre (10, 8)`	Un commentaire : une phrase pour celui qui lit (« 11. un pas en diagonale : retour au centre (10, 8) »). La console l'ignore.
`carre(10, 8, ALPHABET[0], 1, 1, 250, 1);`	Appel de la fonction carre de la console, avec les arguments 10, 8, ALPHABET[0], 1, 1, 250, 1. Ce qu'elle fait : un carré parfait AUTOUR de (x, y) : taille 1 fait 3 × 3, taille 2 fait 5 × 5 (2 × taille + 1). sens 1 : aiguilles d'une montre, -1 : l'autre sens. vitesse : millisecondes par pas, écrite en clair (250). La lettre part du centre, fait ses tours et y revient. Ou, les sept réglages sous un nom : Carre ronde = { 10, 8, ALPHABET[0], 2, 1, 250, 3 }; carre(ronde);.
`// Ici, carre a fini : le A est revenu au centre (10, 8), et il y reste.`	Un commentaire : une phrase pour celui qui lit (« Ici, carre a fini : le A est revenu au centre (10, 8), et il y reste. »). La console l'ignore.
`// while (true) tourne sans fin, pour que le programme ne s'arrête jamais.`	Un commentaire : une phrase pour celui qui lit (« while (true) tourne sans fin, pour que le programme ne s'arrête jamais. »). La console l'ignore.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();  // attend l'image suivante (60 par seconde)`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « attend l'image suivante (60 par seconde) ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1349 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  carre()         440 o  ← main()  — une fonction fournie par le langage
  main()           38 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← poser(), effacer()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
```

```
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()   × 4    52 o sur place, dans carre()  + AdresseCase (10 o)
  effacer() × 3    45 o sur place, dans carre()  + AdresseCase (10 o), EffacerCases (12 o)
  image()   × 4    12 o sur place, dans carre(), main()  + AttendreImage (28 o)
Tes fonctions appelées :
  carre()   × 1    31 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// main : le programme commence ici. Les lignes entre { et } s'exécutent
// dans l'ordre, de haut en bas.

#include <carre>      // une tuile qui tourne en carré
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

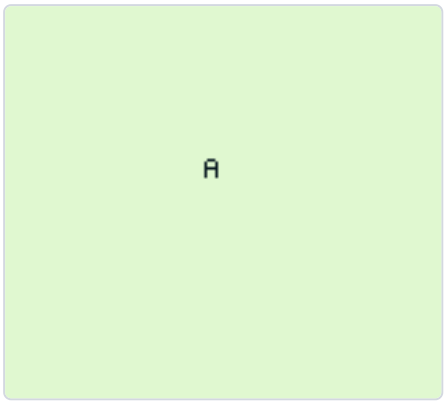
```
int main() {
  // Une seule ligne fait tout le carré. Ses sept réglages, dans l'ordre :
  //
  //   carre(10, 8, ALPHABET[0], 1, 1, 250, 1);
  //         |   |  |           |  |  |    |
  //         |   |  |           |  |  |    +-- tours   : 1, un seul tour
  //         |   |  |           |  |  +------- vitesse : 250 millisecondes par pas,
  //         |   |  |           |  |                     soit 4 pas par seconde
  //         |   |  |           |  +---------- sens    : 1, comme les aiguilles
  //         |   |  |           |                        d'une montre
  //         |   |  |           +------------- taille  : 1, le plus petit carré (3 × 3)
  //         |   |  +--------------------------- tuile   : ce qui tourne ; ALPHABET[0]
  //         |   |                                       est la 1re lettre, A
  //         |   +------------------------------ y       : la LIGNE du centre, 8
  //         +---------------------------------- x       : la COLONNE du centre, 10
  //
  // Pour ce premier cours, regarde surtout x, y et la tuile. Les quatre
  // autres réglages seront changés un par un dans les cours suivants.
  //
  // Ce que fait le A, un pas tous les quarts de seconde :
  //
  //      colonne :   9    10    11
  //      ligne 7 :   2  →  3  →  4
  //                  ↑          ↓
  //      ligne 8 :   9     A     5
  //                  ↑          ↓
  //      ligne 9 :   8  ←  7  ←  6
  //
  //   1.     le A apparaît au centre (10, 8)
  //   2.     un pas en diagonale : le coin en haut à gauche (9, 7)
  //   3 à 9. le tour, une case à la fois : droite, bas, gauche, haut…
  //   10.    …jusqu'au coin (9, 7), d'où il était parti
  //   11.    un pas en diagonale : retour au centre (10, 8)
  carre(10, 8, ALPHABET[0], 1, 1, 250, 1);

  // Ici, carre a fini : le A est revenu au centre (10, 8), et il y reste.
  // while (true) tourne sans fin, pour que le programme ne s'arrête jamais.
  while (true) {
    image();          // attend l'image suivante (60 par seconde)
  }
}
```

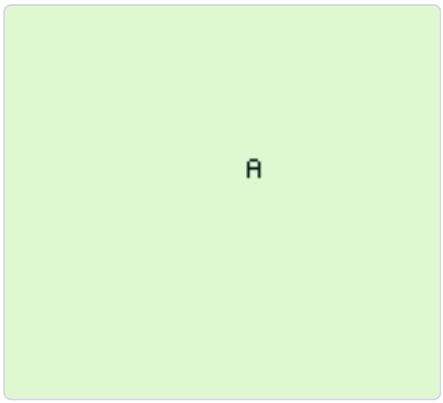
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

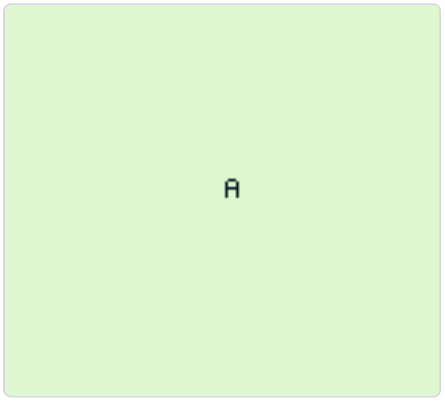
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

> Un A qui fait un petit tour en carré autour de la case (10, 8), puis s'arrête au centre.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les quatre coins, dans le sens des aiguilles d'une montre (9,7 11,7 11,9 9,9)
- ✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.34.1. Un carré autour d'une lettre, avec carre — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++